

sourcetest

November 29, 2017
17:16

Contents

1	Test source sampling routines	1
2	Initialize	17
3	INDEX	18

1 Test source sampling routines

`$Id: c0253cab1da2860cfa37eaf1ca5044052e029abf $`

This program consists of two primary pieces:

1. Test of the sampling of the source segments within a group (called the “segment test”),
2. Test of the sampling of particles from a given source segment (indicated below by “velocity test”).

sourcetest can only be run after the problem and background netCDF files have been generated. Only the sources specified in those files can be tested.

Segment Test

The objective of the segment test is to verify that the distribution of sampled flights corresponds to the input current distribution (i.e., as specified by the input *source_current* array). This test is carried out on only source group at a time. The code asks the user for that source group number as well as the number of samples to be used in the test:

```
Enter index for a source group, 1 to    2 And number of samples
```

The user's response should be a pair of integers, with the first being between the indicated limits (the upper bound being the number of source groups in the problem, *so_grps*). For the second, any number larger than 1 is valid. A useful number would be one comparable to the number of flights expected to be used for that group in the run of DEGAS 2.

The output from *sourcetest* is written to a file in the run directory called **sourcetest.out**.

The first section describes the results of the segment test. For example,

```
For source group =    1
Seg  Ptr    Weight    Rel.Wt.    Rel.Curr.
  1   234  0.0000E+00  0.0000E+00  2.9232E-02
  2   233  5.0000E+00  5.0000E-02  2.7951E-02
  3   232  4.0000E+00  4.0000E-02  2.6836E-02
:
30   205  6.0000E+00  6.0000E-02  4.3854E-02
31   204  5.0000E+00  5.0000E-02  4.5339E-02
32   203  5.0000E+00  5.0000E-02  4.8014E-02
Minimum
Rel.Curr.    Num. Points    Chi Sqd./Num.Pts.    Prob.
1.0E-02           32           0.988628      0.484895
```

The first line identifies the source group. The columns are:

Seg The source segment number. This will just be an enumeration of the segments in the group.

Ptr A pointer from each segment to its corresponding geometry element. For a surface source, the pointer provides a sector number. For a volume source, it corresponds to a zone number.

Weight The total weight sampled from each segment during the test. If the relative weight (*source_segment_rel_wt*) is 1, this is just the number of flights sampled.

Rel.Wt. Is the sampled weight divided by the total weight (the number of samples).

Rel.Curr. Is the current on each segment (*source_current*) divided by the total source current for the group (*so_tot_curr*).

If the sampling routines are working correctly, the **Rel.Wt.** column will smoothly approach the **Rel.Curr.** column as the number of samples approaches infinity. The code attempts to quantify this agreement by evaluating a χ^2 expression (e.g., see Bevington, *Data Reduction and Error Analysis for the Physical Sciences*),

$$\chi^2 = \sum_{i=1}^{N_{\text{seg}}} \frac{(w_i - w_{\text{tot}} p_i)^2}{w_{\text{tot}} p_i}, \quad (1)$$

where N_{seg} is the number of source segments, w_i is the weight sampled on the i th segment (**Weight** column), w_{tot} is the total weight sampled (equals the number of samples), and p_i is the relative current (column **Rel.Curr.**) for the segment. The denominator represents the uncertainty in the sample for the i th segment (the variance). Since this process represents the random sampling of a finite number of items, the results should have a Poisson distribution and the variance should equal the mean, $w_{\text{tot}} p_i$. The code actually prints out the reduced $\chi^2_{\nu} \equiv \chi^2 / N_{\text{seg}}$.

The cumulative probability distribution of χ^2 , $P_{\chi}(\chi^2 | \nu)$ is given by

$$P_{\chi}(\chi^2 | \nu) = 1 - \mathcal{P}\left(\frac{\nu}{2}, \frac{\chi^2}{2}\right), \quad (2)$$

where

$$\mathcal{P}(a, x) \equiv \frac{1}{\Gamma(a)} \int_0^x e^{-t} t^{a-1} dt, \quad (3)$$

is the incomplete gamma function, and Γ is the gamma function. The symbol $\mathcal{P}$ was chosen for the incomplete gamma function so as to be consistent with the notation in Abramowitz and Stegun and is obviously a possible source of confusion in Eq. (2). Equation (2) for $P_{\chi}(\chi_0^2 | \nu)$ gives the probability that a typical sampling of the source distribution will yield a χ^2 , call it X^2 , with $X^2 > \chi_0^2$. In other words, if there is a problem with the sampling routines, you will consistently find small values for this probability $P_{\chi}(\chi_0^2 | \nu)$. Note that for $\nu \gg 1$, $\chi_{\nu}^2 = 1$ corresponds roughly to a probability of 0.5. So, values of $P_{\chi}(\chi_0^2 | \nu) \simeq 0.5$ do *not* indicate a problem. If χ_{ν}^2 is significantly larger than 1, say, > 1.5 , or if the resulting probability is very small, the sampling process should be examined more carefully. On the other hand, if the sampling routine “nails” the distribution, $P_{\chi}(\chi_0^2 | \nu)$ will approach one. This is what you would obtain when using the “direct sampling” method described in *sources.hweb*. In this case, the error scales like $1/w_{\text{tot}}$ rather than $1/\sqrt{w_{\text{tot}}}$. The result will be small values of χ_{ν}^2 , decreasing steadily with increasing w_{tot} . The probabilities in this case quickly reach 1 as w_{tot} grows.

Additional machinery has been added to *sourcetest* to aid in evaluating the sampling of recombination sources. Because the code automatically assumes that every plasma zone is a source, no matter how great, the range in source currents can be many orders of magnitude. The result is that any reasonable sample size will still leave a significant number of segments unsampled. These unsampled, physically irrelevant, segments can dominate the value of χ_{ν}^2 . To better evaluate the sampling, several χ_{ν}^2 are computed by restricting the analysis to zones having a relative current greater than some minimum value, p_{min} . For example, for $p_{\text{min}} = 10^{-4}$, a χ_{ν}^2 is computed based on only those segments having $p_i \geq 10^{-4}$; the number of such segments is given as well. The resulting values and probabilities can be interpreted as described above, but the user’s attention should be focussed on the higher values for the minimum relative current. These represent the zones which will be the physically significant sources in the problem.

Velocity Test

The purpose of this part of *sourcetest* is to check the velocity distribution of the neutrals generated by this source. It also provides the spatial distribution of the source particles across the specified source segment. The random number generator is called in such a way that the random number sequence should exactly match that used in the main code. Hence, if there is only one source segment in the problem, the particle positions and velocities produced here should be exactly the same as those that will be used to initialize the flights in the run of *flighttest*.

Note that there is no on-screen feedback following the segment test, except in the case of code failure. Instead, the user is immediately presented with the velocity test. A choice for the source group to be examined is requested:

```
Segment test: enter source group, 1 to    2
```

The user is asked to choose a particular segment (sector or zone, depending on the type of source) of this group and to enter the number of samples:

```
Enter segment number, 1 to    32, and number of samples
```

Following the segment test output in *sourcetest.out*, the code will write the output for the velocity test. In this case, *there will be one line per sample*, so be careful when specifying the number of samples!

The first line of this output section echoes the input information and provides a pointer to the corresponding geometry element, a sector number for surface sources, zone number for volume sources:

```
For source group    1, segment    25, which points to geometry element    210
```

The actual output differs for surface and volume sources. Here is an example of the output for a surface source:

i	x energy	y fr	z cos_th	vx vpar1	vy vpar2	vz fr_y
0	1.0000E-08	1.9390E-01	1.2678E-01	2.1346E+03	-3.2493E+02	
2.0787E+02	9.8218E-02	2.8451E-01	9.8406E-01	9.5827E-02	1.4980E-01	
7.5591E-01						
1	1.0000E-08	1.7510E-01	1.2757E-01	3.9551E+02	-6.7830E+02	
-4.3682E+00	1.2869E-02	4.1078E-01	5.0371E-01	-5.5632E-03	8.6386E-01	
3.9839E-03						
2	1.0000E-08	1.9999E-01	1.2889E-01	3.9319E+02	-1.6741E+03	
4.5871E+02	6.6119E-02	6.2271E-01	2.2092E-01	2.5773E-01	9.4062E-01	
9.9943E-01						
3	1.0000E-08	1.7604E-01	1.2871E-01	6.1353E+02	-3.8940E+02	
-4.5450E+02	1.5334E-02	5.9407E-01	7.1582E-01	-5.3027E-01	4.5432E-01	
4.1716E-02						
:						
:						

Where

i Is just a label for each individual sample. Again, if there is only one source segment in the problem, this position and velocity will be that of flight number *i* in *flighttest*.

x,y,z Are the coordinates in physical space. These should lie within the bounds of the chosen sector.

vx,vy,vz Are the components of the velocity. Note that in some cases these can be quite simple. For example, the usual assumption for a plate source is normal incidence. I.e., this initial vector will be normal to the sector.

energy The sampled energy. In some cases, such as the EIRENE benchmark examples, this may be constant.

fr This is the fractional distance of the sample from the beginning of the sector computed using the values of *sector_points*; should be between 0 and 1. The distribution of this quantity can be plotted to verify the spatial sampling scheme.

cos.th Cosine of the angle of incidence relative to the surface normal. For the usual plate source, this will be -1, with the minus sign indicating that the flight is directed towards the surface.

vpar1 Component of the sampled velocity parallel to the the line formed by the endpoints of the sector (in *sector_points*).

vpar2 Second parallel component of the sampled velocity, perpendicular to the endpoints of the sector.

fr_y For 3-D cases, this is the fractional distance of the sample from the beginning of the sector in the *y* (or in toroidal angle in cylindrical / toroidal problems). This is computed using the *zone_min* and *zone_max* of the source segment's *sector_zone*. In 2-D cases, this will be zero.

The heading used for a volume source is:

i	x	y	z	vx	vy	vz
	energy	fr	vhatx	vhaty	vhatz	fr_y

The *fr* and *fr_y* quantities will be zero in this case. The *vhatx*, etc. columns provide the unit vector for the velocity, $\vec{v}/v$.

At the end of the file, averages of these quantities are given. These can be used as a quick check of the sampled positions and velocities. However, testing of new source routines should include plotting the distributions of these velocities and computing moments higher than the mean (e.g., the variance).

```
"sourcetest.f" 1 ≡
  @m FILE 'sourcetest.web'
```

The main program.

```
"sourcetest.f" 1.1 ≡
  program sourcetest
    implicit none_f77
    implicit none_f90

    call readfilenames
    call init_sourcetest
    call do_src_tests

    stop
  end
```

⟨ Functions and subroutines 1.2 ⟩

Routine to actually do tests.

```
"srcettest.f" 1.2 ≡
  @m max_num_chis 10
  @f sources_localcommon integer
  @m sources_localcommon save/sources_common/

      vc_decl(x_surface) common/sources_common/ x_surface

⟨ Functions and subroutines 1.2 ⟩ ≡
subroutine do_src_tests
  define_varp(stat, FLOAT, source_seg_ind)
  define_varp(prob_seg, FLOAT, source_seg_ind)

  implicit_none_f77
  gi_common
  zn_common
  sp_common
  sc_common
  so_common
  implicit_none_f90
  real rel_wt, weight_tot, energy, fr, vsrc_mag, vnorm_mag, cos_theta, deltax_mag, vpar1, vpar2,
      vplane2_mag, phi, y_min, y_max, fr_y, min_prob, max_prob, energy_avg, fr_avg, costh_avg,
      vpar1_avg, vpar2_avg, fr_y_avg, dummy
  real chi_sqd max_num_chis, estimator_factors_1
  integer num, i, xseg, nsample, zone, nseg, j, num_chis, kseg, iparam
  integer num_points max_num_chis
  integer length, p, b, e
  character*LINELEN line

  external gammq      // External
  real gammq

  rn_seed_decl(seed)
  rn_seed_decl(temp_seed)
  rn_decl(temp_rand)
  lc_decl(xsrc)
  so_decl(isource)
  sp_decl(spsrc)
  fl_decl(x)
  vc_decl(vzero)
  vc_decl(vone)
  vc_decl(deltax)
  vc_decl(xp)
  vc_decl(vsrc)
  vc_decl(xsrc_avg)
  vc_decl(vsrc_avg)
  vc_decl(vnorm)
  vc_decl(vplane2)
  vc_decl(s0)
  vc_decl(s1)

sources_localcommon

⟨ Memory allocation interface 0 ⟩
  rn_decls
```

```

    st_decls
    pt_decls
    lc_decls
    vc_decls

    declare_varp(stat)
    declare_varp(prob_seg)

    var_alloc(stat)
    var_alloc(prob_seg)

@#if 0
    rn_init(seed, fl_rand(x))    // Initialize random numbers for flight
@#endif
    call decimal_to_seed(so_seed_decimal, rn_seed_args(seed))
    if (so_sampling ≡ so_direct) then
        rn_seed_copy(seed, temp_seed)
        call next_seed(-1, rn_seed_args(temp_seed))
        rn_init(rn_seed_args(temp_seed), temp_rand)
        rn_next(so_direct_delta, temp_rand)
    else if (so_sampling ≡ so_random) then
        so_direct_delta = zero
    else
        assert('Illegal_value_of_so_sampling' ≡ '_')
    end if

    open(unit = diskout, file = 'sourcetest.out', status = 'unknown')

loop1: continue

    write(stdout,
        *) 'Enter_index_for_a_source_group, 1 to ', so_grps, ' And_number_of_samples'
    if (¬read_string(stdin, line, length))
        goto break
    assert(length ≤ len(line))
    length = parse_string(line(: length))
    p = 0
    assert(next_token(line, b, e, p))
    isource = read_integer(line(b : e))
    if (¬so_check(isource)) then
        write(stdout, *) 'Source_group', isource, 'is_invalid; try again.'
        go to loop1
    end if
    assert(next_token(line, b, e, p))
    nsample = read_integer(line(b : e))

    if (so_nseg(isource) > 1) then
        min_prob = one
        max_prob = zero
        do i = 1, so_nseg(isource)
            prob_seg_i = source_currentso_base(isource)+i-1 / so_tot_curr(isource)
            if (prob_seg_i > zero) then
                min_prob = min(min_prob, prob_seg_i)
                max_prob = max(max_prob, prob_seg_i)
            end if
        end do
    end do

```



```

    assert(min_prob > zero)
    assert(max_prob < one)

    do i = 1, so_nseg(isource)
        stat_i = zero
    end do

    weight_tot = zero

@#if 0
    do num = 1, nsample
@#endif
        do num = 0, nsample - 1
            rn_init(seed, fl_rand(x))
            fl_label(x, num, isource)
            call sample_sources(tl_est_unknown, fl_args(x), estimator_factors, rel_wt)
@#if 0
            call sample_sources(fl_args(x), rel_wt)
@#endif
            stat_fl_source_seg(x)+1 += rel_wt
            weight_tot += rel_wt
            call next_seed(1, seed)
        end do

        min_prob = const(1., 1)max(int(log10(min_prob))-1, -10)
        num_chis = int(log10(max_prob / min_prob)) + 1
        assert(num_chis ≤ max_num_chis)
        do j = 1, num_chis
            chi_sqd_j = zero
            num_points_j = 0
        end do

        write(diskout, *)
        write(diskout, *) '___For_source_group= ', isource
        write(diskout, *) '___Seg___Ptr___Weight___Rel.Wt. ___Rel.Curr.'
        do i = 1, so_nseg(isource)
            write(diskout,
                '(i5,1x,i5,1x,1pe11.4,1x,e11.4,1x,e11.4)') i, source_segment_ptr_so_base(isource)+i-1,
                stat_i, stat_i / weight_tot, prob_seg_i
            do j = 1, num_chis
                if (prob_seg_i > min_prob * const(1., 1)j-1) then
                    num_points_j++
                    chi_sqd_j += (stat_i - weight_tot * prob_seg_i)2 / (weight_tot * prob_seg_i)
                end if
            end do
        end do

        write(diskout, *) '___Minimum'
        write(diskout, *) '___Rel.Curr. ___Num. ___Points ___Chi_Sqd. / Num.Pts. ___Prob.'
        do j = 1, num_chis
            assert(num_points_j > 0)
            write(diskout, '(2x,1pe8.1,7x,i5,8x,e13.6,5x,e13.6)') min_prob * const(1., 1)j-1,
                num_points_j, chi_sqd_j / num_points_j, gammq(areal(num_points_j) * half, chi_sqd_j * half)
        end do
    end if

```

```

break: continue

    var_free(stat)
    var_free(prob_seg)

loop2: continue

    write(stdout, *) 'Segment_test: enter source_group, 1 to ', so_grps
    if (¬read_string(stdin, line, length))
        goto eof
    assert(length ≤ len(line))
    length = parse_string(line(: length))
    p = 0
    assert(next_token(line, b, e, p))
    isource = read_integer(line(b : e))
    if (¬so_check(isource)) then
        write(stdout, *) 'Source_group', isource, 'is invalid; try again.'
        go to loop2
    end if
loop3: continue
    write(stdout,
        *) 'Enter segment_number, 1 to ', so_nseg(isource), ', and number of samples'
    if (¬read_string(stdin, line, length))
        goto eof
    assert(length ≤ len(line))
    length = parse_string(line(: length))
    p = 0
    assert(next_token(line, b, e, p))
    nseg = read_integer(line(b : e))
    if (nseg ≤ 0 ∨ nseg > so_nseg(isource)) then
        write(stdout, *) 'Segment', nseg, 'is invalid; try again.'
        go to loop3
    end if
    assert(next_token(line, b, e, p))
    nsample = read_integer(line(b : e))

    xseg = source_segment_ptrso_base(isource)+nseg-1
    kseg = int_lookup(xseg, source_segment_ptrso_base(isource), so_nseg(isource)) - 1

    vc_set(vzero, zero, zero, zero)
    vc_set(vone, zero, zero, one)

    write(diskout, *)
    write(diskout, '(a,i2,a,i5,a,i5)') 'For source_group', isource, ', segment', nseg,
        ', which points to geometry element', xseg
    if (so_geom(isource) ≠ so_volume) then
        write(diskout, *) ' i x y z \
            vx vy vz energy fr cos_th \
            vpar1 vpar2 fr y'
    else
        write(diskout, *) ' i x y z \
            vx vy vz energy fr vhatx \
            vhaty vhatz fr y'
    end if
    vc_set(xsrc_avg, zero, zero, zero)

```

```

    vc_set(vsrc_avg, zero, zero, zero)
    energy_avg = zero
    fr_avg = zero
    costh_avg = zero
    vpar1_avg = zero
    vpar2_avg = zero
    fr_y_avg = zero    /* Try to sync these samples with a normal run, too. */
    call decimal_to_seed(so_seed_decimal, rn_seed_args(seed))

@#if 0
    do i = 1, nsample
@#else
    do i = 0, nsample - 1
@#endif
    rn_init(seed, fl_rand(x))
    if (so_sampling ≡ so_random) then
        rn_next(dummy, fl_rand(x))    // Match one used for segment choice.
    end if
    call set_source_x(tl_est_unknown, so_args(isource), xseg, kseg, rn_args(fl_rand(x)),
        lc_args(xsrc))
    call set_source_v(so_args(isource), xseg, kseg, lc_args(xsrc), rn_args(fl_rand(x)),
        vc_args(vsrc))
    call next_seed(1, seed)
    if (so_type(isource) ≠ so_snapshot) then
        spsrc = so_species(isource)
    else
        call init_snapshot_fl(so_args(isource), xseg, kseg, fl_args(x))
        spsrc = pt_sp(fl_origin(x))
    end if
    energy = half * sp_m(spsrc) * vc_abs2(vsrc) / electron_charge
    /* For surface sources, break xsrc and vsrc down into components relative to surface. */
    if (so_geom(isource) ≡ so_surface) then
        /* Use x_surface set in set_source_x, just as in main code. */
        call surface_reflect(sector_surface_xseg, vc_args(x_surface), vc_args(vzero), vc_args(vone),
            vc_args(vnorm))
        vnorm_mag = vc_abs(vnorm)
        vsrc_mag = vc_abs(vsrc)
        if (vnorm_mag ≠ zero ∧ vsrc_mag ≠ zero) then
            cos_theta = vc_product(vnorm, vsrc) / (vnorm_mag * vsrc_mag)
        else
            cos_theta = zero
        end if    /* Rotate sector_points to same toroidal angle as x_surface, if needed. */
        if ((geometry_symmetry ≡ geometry_symmetry_cyl_hw) ∨ (geometry_symmetry ≡
            geometry_symmetry_cyl_section)) then
            phi = atan2(x_surface_2, x_surface_1)    /* This is assumed in set_source_x also. */
            assert(sector_points_xseg,0,2 ≡ zero)
            assert(sector_points_xseg,1,2 ≡ zero)
            vc_set(s0, sector_points_xseg,0,1 * cos(phi), sector_points_xseg,0,1 * sin(phi),
                sector_points_xseg,0,3)
            vc_set(s1, sector_points_xseg,1,1 * cos(phi), sector_points_xseg,1,1 * sin(phi),
                sector_points_xseg,1,3)    /* Fractional distance in y. */
            y_min = zone_min_sector_zone_xseg,2
            y_max = zone_max_sector_zone_xseg,2

```

```

    if (phi < y_min) then
        phi += two * PI
    else if (phi > y_max) then
        phi -= two * PI
    end if
    fr_y = (phi - y_min) / (y_max - y_min)
    assert((fr_y ≥ zero) ∧ (fr_y ≤ one))
else
    vc_copy(sector_pointsxseg,0, s0)
    vc_copy(sector_pointsxseg,1, s1)
    if (geometry_symmetry ≡ geometry_symmetry_plane_hw) then
        y_min = zone_minsector_zonexseg,2
        y_max = zone_maxsector_zonexseg,2
        fr_y = (x_surface2 - y_min) / (y_max - y_min)
        assert((fr_y ≥ zero) ∧ (fr_y ≤ one))
    else
        fr_y = zero
    end if
end if /* Distance along segment, all symmetries */
vc_difference(s1, s0, deltax)
vc_difference(x_surface, s0, xp)
fr = vc_product(xp, deltax) / vc_abs2(deltax)
assert(fr ≥ zero ∧ fr ≤ one)

// Components in plane of surface
deltax_mag = vc_abs(deltax) // deltax is one direction in plane
if (deltax_mag ≠ zero ∧ vsrc_mag ≠ zero) then
    assert(abs(vc_product(deltax, vnorm) / (deltax_mag * vnorm_mag)) < geom_epsilon)
    // Check that normal is normal
    vpar1 = vc_product(deltax, vsrc) / (deltax_mag * vsrc_mag)
else
    vpar1 = zero
end if

vc_cross(vnorm, deltax, vplane2) // A second direction in plane
vplane2_mag = vc_abs(vplane2)
if (vplane2_mag ≠ zero ∧ vsrc_mag ≠ zero) then
    vpar2 = vc_product(vplane2, vsrc) / (vplane2_mag * vsrc_mag)
else
    vpar2 = zero
end if
else if (so_geom(isource) ≡ so_volume) then
    fr = zero
    fr_y = zero
    vsrc_mag = vc_abs(vsrc)
    if (vsrc_mag > zero) then
        cos_theta = vsrc1 / vsrc_mag
        vpar1 = vsrc2 / vsrc_mag
        vpar2 = vsrc3 / vsrc_mag
    else
        cos_theta = zero
        vpar1 = zero
        vpar2 = zero
    end if
end if

```

```

        end if
    else    // Not a surface or volume source
        fr = zero
        cos_theta = zero
        vpar1 = zero
        vpar2 = zero
        fr_y = zero
    end if

    write(diskout, '(1x,i5,1x,1p,12(e11.4,2x))') i, lc_x(xsrc)_1, lc_x(xsrc)_2, lc_x(xsrc)_3,
        vsrc_1, vsrc_2, vsrc_3, energy, fr, cos_theta, vpar1, vpar2, fr_y
    vc_add(lc_x(xsrc), xsrc_avg, xsrc_avg)
    vc_add(vsrc, vsrc_avg, vsrc_avg)
    energy_avg += energy
    fr_avg += fr
    costh_avg += cos_theta
    vpar1_avg += vpar1
    vpar2_avg += vpar2
    fr_y_avg += fr_y
end do
write(diskout, *) '-----\
-----\
-----'
vc_scale(one / areal(nsampl), xsrc_avg, xsrc_avg)
vc_scale(one / areal(nsampl), vsrc_avg, vsrc_avg)
energy_avg /= areal(nsampl)
fr_avg /= areal(nsampl)
costh_avg /= areal(nsampl)
vpar1_avg /= areal(nsampl)
vpar2_avg /= areal(nsampl)
fr_y_avg /= areal(nsampl)
write(diskout, *) 'Averages:'
write(diskout, '(7x,1p,12(e11.4,2x))') xsrc_avg_1, xsrc_avg_2, xsrc_avg_3, vsrc_avg_1, vsrc_avg_2,
    vsrc_avg_3, energy_avg, fr_avg, costh_avg, vpar1_avg, vpar2_avg, fr_y_avg
@if 0
    go to loop2
@endif
eof: continue
close(unit = diskout)

return
end

```

See also sections 1.3, 1.4, 1.5, 1.6, and 2.

This code is used in section 1.1.

Incomplete gamma function. Returns the incomplete gamma function

$$Q(a, x) \equiv \frac{1}{\Gamma(a)} \int_x^\infty e^{-t} t^{a-1} dt, \quad (4)$$

for $a > 0$. Taken from *Numerical Recipes* by W. H. Press et al. (Cambridge Press, New York, 1986).

⟨ Functions and subroutines 1.2 ⟩ +≡

```
function gammq(a, x)
    implicit_none_f77
    implicit_none_f90

    real gammq    // Function

    real a, x     // Input

    real gamser, gln, gammcf    // Local

    assert(x ≥ 0 ∧ a > 0)
    if (x < a + one) then
        call gser(a, x, gamser, gln)    // Use series representation
        gammq = one - gamser
    else
        call gcf(a, x, gammcf, gln)    // Use continued fraction
        gammq = gammcf
    end if

    return
end
```

Series representation of the incomplete gamma function. Returns the incomplete gamma function $P(a, x)$ evaluated by its series representation as *gamser*. Also returns $\ln \Gamma(a)$ as *gln*. Taken from *Numerical Recipes* by W. H. Press et al. (Cambridge Press, New York, 1986).

```
"sourcetest.f" 1.4 ≡
  @m itmax 500
  @m eps const(3., -7)

⟨ Functions and subroutines 1.2 ⟩ +≡
  subroutine gser(a, x, gamser, gln)

    implicit_none_f77
    implicit_none_f90

    real a, x      // Input
    real gamser, gln // Output

    integer n      // Local
    real ap, sum, del

    external gammln // External
    real gammln

    gln = gammln(a)
    if (x ≤ zero) then
      assert(x ≡ zero)
      gamser = zero
      return
    end if
    ap = a
    sum = one / a
    del = sum
    do n = 1, itmax
      ap += one
      del *= x / ap
      sum += del
      if (abs(del) < abs(sum) * eps)
        go to break
      end do
      assert("a_too_large, itmax_too_small" ≡ "")
break: continue
    gamser = sum * exp(-x + a * log(x) - gln)

    return
  end
```

Continued fraction representation of the incomplete gamma function. Returns the incomplete gamma function $Q(a, x)$ evaluated by its continued fraction representation as *gammcf*. Also returns $\ln \Gamma(a)$ as *gln*. Taken from *Numerical Recipes* by W. H. Press et al. (Cambridge Press, New York, 1986).

⟨ Functions and subroutines 1.2 ⟩ +≡

```

subroutine gcf(a, x, gammcf, gln)

    implicit_none_f77
    implicit_none_f90

    real a, x    // Input
    real gammcf, gln    // Output

    integer n    // Local
    real a0, a1, b0, b1, fac, an, ana, anf, gold, g

    external gammln    // External
    real gammln

    gln = gammln(a)    /* This is the previous value, tested against for convergence. */
    gold = zero    /* Setting the A's and B's of Eq. (5.2.4) in Numerical Recipes for evaluating the
        continued fraction. */
    a0 = one
    a1 = x
    b0 = zero
    b1 = one    /* fac is the renormalization factor for preventing overflow of the partial numerators
        and denominators. */
    fac = one
    do n = 1, itmax
        an = areal(n)
        ana = an - a
        a0 = (a1 + a0 * ana) * fac    // One step of the recurrence (5.2.5)
        b0 = (b1 + b0 * ana) * fac
        anf = an * fac
        a1 = x * a0 + anf * a1    // The next step of the recurrence (5.2.5)
        b1 = x * b0 + anf * b1
        if (a1  $\neq$  zero) then
            fac = one / a1    // Renormalize
            g = b1 * fac    // New value of answer
            if ((abs(g - gold) / g) < eps)
                go to break    // Is converged.
            gold = g    // Not converged, try again
        end if
    end do
    assert("a_too_large, itmax_too_small"  $\equiv$  "")
break: continue
    gammcf = g * exp(-x + a * log(x) - gln)

    return
end

```


Log of the gamma function. Returns the value of $\ln \text{Gamma}(x)$ for $x > 0$. Full accuracy is obtained for $x > 1$. For $x < 1$, the reflection formula is used. This routine is taken from *Numerical Recipes* by W. H. Press et al. (Cambridge Press, New York, 1986).

⟨ Functions and subroutines 1.2 ⟩ +≡

```

function gammaln(xx)

    implicit_none_f77
    implicit_none_f90

    real gammaln    // Function

    real xx          // Input

    integer j, smallx // Local
    real stp, x, tmp, ser
    real cof(6)

    cof(1) = const(76.18009173)
    cof(2) = const(-86.50532033)
    cof(3) = const(24.01409822)
    cof(4) = const(-1.231739516)
    cof(5) = const(0.120858003, -2)
    cof(6) = const(-0.536382, -5)

    stp = const(2.50662827465)

    assert(xx > zero)
    if (xx < one) then
        smallx = TRUE    // To use reflection formula Eq. (6.1.4)
        x = one - xx
    else
        smallx = FALSE
        x = xx - one
    end if
    tmp = x + const(5.5)
    tmp = (x + half) * log(tmp) - tmp

    ser = one
    do j = 1, 6
        x += one
        ser += cof(j) / x
    end do

    gammaln = tmp + log(stp * ser)
    if (smallx ≡ TRUE)
        gammaln = log(PI * (one - xx) / sin(PI * (one - xx))) - gammaln

    return
end

```

2 Initialize

⟨ Functions and subroutines 1.2 ⟩ +≡

```
subroutine init_sourcetest
  implicit_none_f77
  implicit_none_f90

  call read_geometry
  call nc_read_elements
  call nc_read_species
  call nc_read_reactions
  call nc_read_materials
  call nc_read_pmi
  call nc_read_problem
  call nc_read_background

  call read_cramd_data

  return
end
```

3 INDEX

- a*: [1.3](#), [1.4](#), [1.5](#).
- abs*: [1.2](#), [1.4](#), [1.5](#).
- an*: [1.5](#).
- ana*: [1.5](#).
- anf*: [1.5](#).
- ap*: [1.4](#).
- areal*: [1.2](#), [1.5](#).
- assert*: [1.2](#), [1.3](#), [1.4](#), [1.5](#), [1.6](#).
- atan2*: [1.2](#).
- a0*: [1.5](#).
- a1*: [1.5](#).
- b*: [1.2](#).
- break*: [1.2](#), [1.4](#), [1.5](#).
- b0*: [1.5](#).
- b1*: [1.5](#).
- chi_sqd*: [1.2](#).
- cof*: [1.6](#).
- const*: [1.2](#), [1.4](#), [1.6](#).
- cos*: [1.2](#).
- cos_theta*: [1.2](#).
- costh_avg*: [1.2](#).
- decimal_to_seed*: [1.2](#).
- declare_varp*: [1.2](#).
- define_varp*: [1.2](#).
- del*: [1.4](#).
- deltax*: [1.2](#).
- deltax_mag*: [1.2](#).
- diskout*: [1.2](#).
- do_src_tests*: [1.1](#), [1.2](#).
- dummy*: [1.2](#).
- e*: [1.2](#).
- electron_charge*: [1.2](#).
- energy*: [1.2](#).
- energy_avg*: [1.2](#).
- eof*: [1.2](#).
- eps*: [1.4](#), [1.5](#).
- estimator_factors*: [1.2](#).
- exp*: [1.4](#), [1.5](#).
- fac*: [1.5](#).
- FALSE*: [1.6](#).
- file*: [1.2](#).
- FILE*: [1](#).
- fl_args*: [1.2](#).
- fl_decl*: [1.2](#).
- fl_label*: [1.2](#).
- fl_origin*: [1.2](#).
- fl_rand*: [1.2](#).
- fl_source_seg*: [1.2](#).
- flighttest*: [1](#).
- FLOAT*: [1.2](#).
- fr*: [1](#), [1.2](#).
- fr_avg*: [1.2](#).
- fr_y*: [1](#), [1.2](#).
- fr_y_avg*: [1.2](#).
- g*: [1.5](#).
- gammcf*: [1.3](#), [1.5](#).
- gammln*: [1.4](#), [1.5](#), [1.6](#).
- gammq*: [1.2](#), [1.3](#).
- gamser*: [1.3](#), [1.4](#).
- gcf*: [1.3](#), [1.5](#).
- geom_epsilon*: [1.2](#).
- geometry_symmetry*: [1.2](#).
- geometry_symmetry_cyl_hw*: [1.2](#).
- geometry_symmetry_cyl_section*: [1.2](#).
- geometry_symmetry_plane_hw*: [1.2](#).
- gi_common*: [1.2](#).
- gln*: [1.3](#), [1.4](#), [1.5](#).
- gold*: [1.5](#).
- gser*: [1.3](#), [1.4](#).
- half*: [1.2](#), [1.6](#).
- hweb*: [1](#).
- i*: [1.2](#).
- implicit_none_f77*: [1.1](#), [1.2](#), [1.3](#), [1.4](#), [1.5](#), [1.6](#), [2](#).
- implicit_none_f90*: [1.1](#), [1.2](#), [1.3](#), [1.4](#), [1.5](#), [1.6](#), [2](#).
- init_snapshot_fl*: [1.2](#).
- init_sourcetest*: [1.1](#), [2](#).
- int*: [1.2](#).
- int_lookup*: [1.2](#).
- integer**: [1.2](#).
- iparam*: [1.2](#).
- isource*: [1.2](#).
- itmax*: [1.4](#), [1.5](#).
- j*: [1.2](#), [1.6](#).
- kseg*: [1.2](#).
- lc_args*: [1.2](#).
- lc_decl*: [1.2](#).
- lc_decls*: [1.2](#).
- lc_x*: [1.2](#).
- len*: [1.2](#).
- length*: [1.2](#).
- line*: [1.2](#).
- LINELEN*: [1.2](#).
- log*: [1.4](#), [1.5](#), [1.6](#).
- log10*: [1.2](#).
- loop1*: [1.2](#).
- loop2*: [1.2](#).

loop3: 1.2.
max: 1.2.
max_num_chis: 1.2.
max_prob: 1.2.
min: 1.2.
min_prob: 1.2.
n: 1.4, 1.5.
nc_read_background: 2.
nc_read_elements: 2.
nc_read_materials: 2.
nc_read_pmi: 2.
nc_read_problem: 2.
nc_read_reactions: 2.
nc_read_species: 2.
next_seed: 1.2.
next_token: 1.2.
nsample: 1.2.
nseg: 1.2.
num: 1.2.
num_chis: 1.2.
num_points: 1.2.
one: 1.2, 1.3, 1.4, 1.5, 1.6.
p: 1.2.
parse_string: 1.2.
phi: 1.2.
PI: 1.2, 1.6.
prob_seg: 1.2.
pt_decls: 1.2.
pt_sp: 1.2.
read_cramd_data: 2.
read_geometry: 2.
read_integer: 1.2.
read_string: 1.2.
readfilenames: 1.1.
rel_wt: 1.2.
rn_args: 1.2.
rn_decl: 1.2.
rn_decls: 1.2.
rn_init: 1.2.
rn_next: 1.2.
rn_seed_args: 1.2.
rn_seed_copy: 1.2.
rn_seed_decl: 1.2.
sample_sources: 1.2.
sc_common: 1.2.
sector_points: 1, 1.2.
sector_surface: 1.2.
sector_zone: 1, 1.2.
seed: 1.2.
ser: 1.6.
set_source_v: 1.2.
set_source_x: 1.2.
sin: 1.2, 1.6.
smallx: 1.6.
so_args: 1.2.
so_base: 1.2.
so_check: 1.2.
so_common: 1.2.
so_decl: 1.2.
so_direct: 1.2.
so_direct_delta: 1.2.
so_geom: 1.2.
so_grps: 1, 1.2.
so_nseg: 1.2.
so_random: 1.2.
so_sampling: 1.2.
so_seed_decimal: 1.2.
so_snapshot: 1.2.
so_species: 1.2.
so_surface: 1.2.
so_tot_curr: 1, 1.2.
so_type: 1.2.
so_volume: 1.2.
source_current: 1, 1.2.
source_seg_ind: 1.2.
source_segment_ptr: 1.2.
source_segment_rel_wt: 1.
sources: 1.
sources_common: 1.2.
sources_localcommon: 1.2.
sourcetest: 1, 1.1.
sp_common: 1.2.
sp_decl: 1.2.
sp_m: 1.2.
spsrc: 1.2.
st_decls: 1.2.
stat: 1.2.
status: 1.2.
stdin: 1.2.
stdout: 1.2.
stp: 1.6.
sum: 1.4.
surface_reflect: 1.2.
s0: 1.2.
s1: 1.2.
temp_rand: 1.2.
temp_seed: 1.2.
tl_est_unknown: 1.2.
tmp: 1.6.
TRUE: 1.6.
two: 1.2.

unit: 1.2.

var_alloc: 1.2.

var_free: 1.2.

vc_abs: 1.2.

vc_abs2: 1.2.

vc_add: 1.2.

vc_args: 1.2.

vc_copy: 1.2.

vc_cross: 1.2.

vc_decl: 1.2.

vc_decls: 1.2.

vc_difference: 1.2.

vc_product: 1.2.

vc_scale: 1.2.

vc_set: 1.2.

vhatx: 1.

vnorm: 1.2.

vnorm_mag: 1.2.

vone: 1.2.

vpar1: 1.2.

vpar1_avg: 1.2.

vpar2: 1.2.

vpar2_avg: 1.2.

vplane2: 1.2.

vplane2_mag: 1.2.

usrc: 1.2.

usrc_avg: 1.2.

usrc_mag: 1.2.

vzero: 1.2.

weight_tot: 1.2.

x: 1.3, 1.4, 1.5, 1.6.

x_surface: 1.2.

xp: 1.2.

xseg: 1.2.

xsrc: 1.2.

xsrc_avg: 1.2.

xx: 1.6.

y_max: 1.2.

y_min: 1.2.

zero: 1.2, 1.4, 1.5, 1.6.

zn_common: 1.2.

zone: 1.2.

zone_max: 1, 1.2.

zone_min: 1, 1.2.

〈Functions and subroutines 1.2, 1.3, 1.4, 1.5, 1.6, 2〉 Used in section 1.1.

〈Memory allocation interface 0〉 Used in section 1.2.

COMMAND LINE: "fweave -f -i! -W[-ybs15000 -ykw800 -ytw40000 -j -n/
/Users/dstotler/degas2/src/sourcetest.web".

WEB FILE: "/Users/dstotler/degas2/src/sourcetest.web".

CHANGE FILE: (none).

GLOBAL LANGUAGE: FORTRAN.